

Plano de Estudos

Preparação para o estágio — Sérgio

Este plano foi montado com base no que você já contou sobre o estágio: o trabalho envolve corrigir bugs e fazer benchmarks de qualidade em sistemas de IA construídos com **LangGraph** e **LangChain**, integrados ao **MLFlow**, rodando em ambiente **Azure** com notebooks Python. O acesso ao Azure só é liberado em outubro, quando as tarefas de fato começam — então você tem cerca de **4 semanas** pra chegar preparado.

A ideia é sair de um domínio sólido em Java/Spring para conseguir ler, debugar e escrever Python com confiança nesse contexto de IA — sem precisar virar especialista em tudo, só o suficiente pra não travar nas primeiras semanas.

Como usar este documento

- **Cronograma** — visão geral das 4 semanas + 1 de reforço.
- **Detalhamento semanal** — o que estudar e como estudar cada tópico, com recursos oficiais.
- **Projetos práticos** — 4 projetos pequenos, do mais simples ao integrador, para praticar fazendo.
- **Autoavaliação** — perguntas para checar se você realmente entendeu cada bloco antes de avançar.
- **Tabela de acompanhamento** — para marcar o que já foi estudado e quando.

Princípio geral de estudo

Para cada tópico novo: primeiro leia o essencial da documentação oficial (não o manual inteiro), depois reproduza um exemplo pequeno com suas próprias mãos, e só então tente o projeto prático correspondente. Evite copiar código pronto de tutoriais em vídeo sem reescrever — o objetivo é conseguir explicar o que cada linha faz.

1. Cronograma geral

Baseado em 1º de setembro como ponto de partida, ajuste as datas se necessário.

Semana	Período	Foco principal	Objetivo da semana
1	01–07 set	Python para IA (retomada)	Escrever código Python limpo e testável
2	08–14 set	Fundamentos de LLMs + LangChain	Montar uma chain simples com LangChain
3	15–21 set	LangGraph (agentes e grafos)	Construir um agente com estado e decisão
4	22–28 set	MLFlow + Observabilidade de IA	Rastrear experimentos e métricas de qualidade
Reforço	29 set–05 out	Azure Notebooks + projeto integrador	Consolidar tudo antes do início oficial

Dica: reserve pelo menos 1h-1h30 por dia útil. É melhor estudar pouco todo dia do que compactar tudo no fim de semana — sistemas de IA e agentes têm muitos conceitos que só grudam com repetição.

2. Detalhamento por semana

Semana 1 — Python para IA (retomada e boas práticas)

O que estudar:

- Funções, list/dict comprehensions, manipulação de arquivos e JSON
- Tratamento de exceções (try/except) — essencial pra a parte de 'corrigir bugs'
- Testes automatizados com pytest (asserts, fixtures básicas)
- Ambientes virtuais (venv) e requirements.txt
- Jupyter Notebooks: células, boas práticas de organização

Como estudar:

- Ler o tutorial oficial do Python (seções de funções, exceções e módulos)
- Resolver 2–3 exercícios pequenos por dia (Exercism ou HackerRank, trilha Python)
- Para cada função que escrever, criar pelo menos 1 teste com pytest

Recursos: docs.python.org/3/tutorial · docs.pytest.org · exercism.org/tracks/python

Semana 2 — Fundamentos de LLMs + LangChain

O que estudar:

- Conceitos-base: prompt, token, temperatura, embeddings, janela de contexto
- LangChain: Chains, PromptTemplates, Output Parsers, Memory
- Integração com um modelo (via API paga ou modelo local com Ollama, pra praticar sem custo)

Como estudar:

- Seguir o tutorial oficial 'Build a simple LLM application' do LangChain
- Recriar 2–3 exemplos da documentação alterando o prompt e a lógica
- Anotar em suas próprias palavras a diferença entre Chain, Prompt e Memory

Recursos: python.langchain.com/docs/get_started · ollama.com

Semana 3 — LangGraph — agentes e grafos de estado

O que estudar:

- Diferença entre um Chain (linear) e um Graph (com estado e ciclos)
- StateGraph, nodes, edges e conditional edges
- Agentes com tool calling (o agente decide qual ferramenta usar)

Como estudar:

- Seguir o Quickstart oficial do LangGraph do início ao fim, rodando cada célula
- Depois de rodar o exemplo, reescrevê-lo mudando os nodes e a condição de decisão
- Desenhar no papel o grafo de estados antes de codar (ajuda a fixar o fluxo)

Recursos: langchain-ai.github.io/langgraph

Semana 4 — MLFlow + Observabilidade de sistemas de IA

O que estudar:

- MLFlow Tracking: experiments, runs, params, metrics, artifacts
- Visão geral do Model Registry
- Observabilidade de IA: latência, custo (tokens), qualidade das respostas, logging de erros

Como estudar:

- Seguir o 'MLflow Tracking Quickstart' oficial rodando localmente
- Registrar métricas de um script simples que chama um LLM (tempo de resposta, tokens)
- Revisar os conceitos de observabilidade que já apareceram no seu processo seletivo — métricas de performance, custo por chamada e qualidade das respostas geradas

Recursos: mlflow.org/docs/latest/tracking.html

Semana de reforço — Azure Notebooks + projeto integrador

- Revisar a documentação e a interface do Azure ML Notebooks (mesmo sem acesso ainda dá pra entender a estrutura: workspaces, compute, notebooks)
- Unir os conceitos das 4 semanas em um projeto pequeno (ver Projeto 4, a seguir)
- Revisar pontos fracos identificados nas autoavaliações de cada semana

3. Projetos práticos

Os projetos abaixo são propostos como **objetivo + estrutura**, não como solução pronta — a ideia é você montar a implementação, já que é assim que o aprendizado gruda melhor. Se travar em algum ponto específico, me chama que a gente resolve juntos.

Projeto 1: Bug Hunter

Objetivo: Praticar debugging estruturado em Python, simulando a parte de 'corrigir bugs' do estágio.

Estrutura sugerida:

- Escreva um script pequeno (ex: calculadora de métricas simples) com 3–4 bugs intencionais que você mesmo insere depois de pronto
- Escreva testes com pytest que devem passar quando o código estiver correto
- Rode os testes, identifique as falhas pelo output, corrija um bug de cada vez

O que isso te ensina: Fixa o hábito de usar testes como guia para encontrar e confirmar correções de bugs.

Projeto 2: Agente simples com LangGraph

Objetivo: Construir seu primeiro agente com decisão condicional.

Estrutura sugerida:

- Um node que recebe uma pergunta do usuário
- Uma conditional edge que decide entre 'responder direto' ou 'buscar informação extra'
- Um segundo node que trata o caminho de busca antes de responder

O que isso te ensina: Entender na prática como um grafo de estados decide o caminho de execução.

Projeto 3: Dashboard de Benchmark

Objetivo: Medir e registrar a performance de chamadas a um LLM.

Estrutura sugerida:

- Um script que roda N perguntas fixas contra um LLM (local ou via API)
- Para cada resposta, medir latência e número de tokens usados
- Registrar cada execução como um 'run' no MLFlow, com os parâmetros e métricas

O que isso te ensina: Simula diretamente a tarefa de 'benchmarks de qualidade de sistemas de IA' do estágio.

Projeto 4: Projeto integrador (semana de reforço)

Objetivo: Juntar tudo: um agente LangGraph monitorado pelo MLFlow.

Estrutura sugerida:

- Reaproveitar o agente do Projeto 2
- A cada execução do agente, registrar no MLFlow: latência, tokens, sucesso/erro
- Adicionar um teste pytest que verifica se o agente sempre retorna uma resposta válida

O que isso te ensina: É o mais próximo do que você provavelmente vai encontrar nas primeiras tarefas reais.

4. Autoavaliação — antes de avançar de semana

Não precisa responder por escrito — é pra você conseguir explicar em voz alta, sem olhar a documentação. Se travar em alguma, é sinal de revisar antes de seguir.

Semana 1 — Python

- Você sabe explicar a diferença entre usar try/except e fazer uma validação prévia com if?
- Consegue escrever um teste pytest do zero para uma função simples?
- Sabe o que é e para que serve um ambiente virtual?

Semana 2 — LangChain

- Sabe explicar o que é um prompt template e por que usá-lo em vez de string fixa?
- Consegue montar uma chain simples do zero, sem olhar exemplo pronto?
- Entende a diferença entre memory de curto e longo prazo numa conversa com LLM?

Semana 3 — LangGraph

- Sabe diferenciar quando usar um Chain e quando usar um Graph?
- Consegue explicar o que é uma conditional edge com suas próprias palavras?
- Sabe descrever como um agente decide qual ferramenta chamar?

Semana 4 — MLFlow e observabilidade

- Sabe o que é um 'run' no MLFlow e o que ele registra?
- Consegue explicar por que monitorar tokens e custo é importante num sistema de IA?
- Consegue listar pelo menos 3 métricas relevantes para observabilidade de sistemas de IA?

5. Tabela de acompanhamento

Use esta tabela para marcar o progresso semana a semana. Pode preencher direto aqui (impresso) ou recriar numa planilha.

Tópico	Status	Data conclusão	Observações
Python — funções e exceções			
Python — testes com pytest			
LangChain — chains e prompts			
LangChain — memory			
LangGraph — StateGraph básico			
LangGraph — agente com tool calling			
MLFlow — tracking de experimentos			
Observabilidade — métricas de qualidade/custo			
Projeto 1 — Bug Hunter			
Projeto 2 — Agente LangGraph			
Projeto 3 — Dashboard de Benchmark			
Projeto 4 — Integrador			
Azure Notebooks — revisão da interface			

Sugestão de status: Não iniciado / Em andamento / Concluído.

6. Recursos oficiais (referência rápida)

Tema	Recurso
Python	docs.python.org/3/tutorial
Testes (pytest)	docs.pytest.org
Prática de exercícios	exercism.org/tracks/python
LangChain	python.langchain.com/docs/get_started
LangGraph	langchain-ai.github.io/langgraph
MLFlow	mlflow.org/docs/latest/tracking.html
Modelo local gratuito	ollama.com
Azure ML (visão geral)	learn.microsoft.com/azure/machine-learning

Boa sorte na preparação, Sérgio. O caminho de Java para esse stack de IA é totalmente viável em 4 semanas com prática consistente — o segredo é priorizar 'fazer' em vez de só ler.